

Failures & Known Issues

Infrastructure Failures, Socket Problems, Gotchas

March 31, 2026

ClawCraft Project Reference

Post-Start Script Failures

Wrong .env file

Notion env vars were written to `~/openclaw/.env` (mounted config dir) instead of `~/src/openclaw/.env` (docker-compose `env_file`). Container couldn't see them in `process.env`. Root cause: docker-compose `env_file`: `.env` reads from the compose directory, not the mounted config path. Fix: added vars to correct file, but this triggered docker compose up which recreated the container and wiped all runtime installs.

Container recreate cascade

Any docker-compose config change requiring `'docker compose up -d'` recreates the container, wiping ALL runtime-installed packages (apt, pip, npm). There is no warning that deps are missing -- things just silently fail. matplotlib imports fail, Playwright can't launch, Slidev isn't found. Fix: always run `docker-post-start.sh` after any compose up.

Duplicate env entries

Writing to `~/openclaw/.env` twice created duplicate `NOTION_*` entries. Harmless (last value wins) but messy. Fix: Python dedup script.

Plugin file ownership

Mac writes plugin files as uid 501. Container runs as uid 1000 (node). After any plugin edit on Mac, container can't read files. Produces cryptic permission errors that don't mention ownership. Fix: post-start script runs `chown -R node:node`. But if you forget to run it after editing, you get silent failures.

jiti cache stale code

Plugin TypeScript is compiled on-the-fly by jiti and cached at `/tmp/jiti/`. After editing `blender-tool.ts` (e.g., adding Notion integration), the OLD cached compiled JS still runs. The new code doesn't take effect. Fix: `rm -f /tmp/jiti/src-blender-*.cjs` or run post-start script. Not obvious when this is the problem -- the code looks right on disk but the runtime is executing old code.

SSL certificate failure

Python 3.14 on Mac throws `CERTIFICATE_VERIFY_FAILED` for all HTTPS requests via `urllib`. The Mac Python install doesn't ship root certificates. Fix: abandoned Python for API calls, used `curl` for all Notion API interactions instead.

zsh reserved variable

Shell function used `'local status="$2"'` but `'status'` is a read-only builtin in zsh. Script failed silently with no error output. Fix: renamed to `'tstatus'`.

Notion API annotation format

Got the annotations placement wrong multiple times. Notion puts 'annotations' as a sibling of the 'text' object, not inside it. Wrong: `{"text": {"content": "bold", "annotations": {"bold": true}}}`. Right: `{"text": {"content": "bold"}, "annotations": {"bold": true}}`. Each failure was a wasted API call.

Notion callout emoji

Used '*' as callout icon. Notion requires a real Unicode emoji. Error response was a massive dump of every valid emoji in the system.

Blender Socket Reliability

Manual start every session

Blender MCP addon must be started from the sidebar (N -> BlenderMCP -> Start Server) each time Blender opens. There is no auto-start option. If you forget, the entire pipeline fails with ECONNREFUSED and no helpful error message.

Socket drops without warning

On March 28: socket was confirmed connected from Docker, but minutes later the agent command got ECONNREFUSED. The addon may have timed out, or Blender may have been in a blocking operation (e.g., a modal dialog, a long script execution). No reconnect logic in the plugin. No error recovery.

No health check or auto-reconnect

Post-start script has a one-shot connectivity test that reports 'connected' or 'not running'. But there's no persistent health check. If the socket drops mid-session (during art production), renders fail silently until someone manually checks. The plugin should reconnect or at least surface a clear error.

host.docker.internal NAT

Docker Desktop maps host.docker.internal to 192.168.65.254 on Mac. The Blender addon binds to 127.0.0.1:9876. Docker Desktop handles the NAT bridge between the container network and the host loopback. This is an extra network hop that occasionally introduces latency or fails entirely.

OpenClaw CLI Issues

message send != agent trigger

'openclaw message send' posts a text message AS the bot to a Discord channel. It does NOT invoke the AI agent. To trigger the agent: 'openclaw agent --to "discord:channel:ID" --message "prompt" --deliver'. This wasn't obvious from the help output.

Agent fallback to embedded mode

When the gateway agent command fails, OpenClaw falls back to an 'embedded' agent that runs in the CLI process. This embedded agent has different tool availability, may not have the same session context, and behaves differently. On March 28, the gateway agent failed, embedded mode tried to reach Blender, failed, and posted an apologetic message to Discord.

sessions.json stale after model switch

Changing the primary model in config does NOT update the cached session's modelProvider field. The bot continues using the old model until you manually nuke sessions.json. Easy to forget, hard to debug -- the config says Sonnet but the bot is still talking to Codex (or vice versa).

Plugin tool name conflict

The blender plugin registers a tool called 'canvas' which conflicts with an internal OpenClaw component. Produces a warning on every startup: 'plugin tool name conflict (blender): canvas'. Cosmetic only, doesn't break functionality, but adds noise to logs.

Retry spiral -- no circuit breaker

One failure in a prior session cascaded into 232 requests, \$5.09, 11.1 million tokens consumed. OpenClaw has no circuit breaker or retry limit. The Codex subscription mitigates text cost (free), but gpt-image-1 (DALL-E) still bills per image. A DALL-E retry spiral could be very expensive.

Current Blocker (March 31)

Zero renders have been completed through the full Discord -> OpenClaw -> Codex -> Blender -> render -> Discord chain. All individual components work and have been tested separately. The TCP socket reliability between Docker and Blender is the single point of failure. The addon must be manually started, drops without warning, and has no auto-reconnect.